Sens-Ex project – Tutorial for processing CMC multimedia data

Drafting: Sangwan Jeon, Ariane Robert, Luisa Troncone
Compilation and translation: Emili Avramovic

This first tutorial aims to explain the data collection methodology according to the different source platforms: Twitch, TikTok, YouTube, and X (Twitter).
Two complementary approaches are to be considered depending on the platforms, the types of data, and the stages of the processing workflow:

- The use of pipelines, which integrate data downloading, timecode-based extraction, and automatic transcription for data from YouTube, TikTok, and Twitch
- The use of free software, for video editing (i.e., extracting relevant segments) and automatic transcription of already downloaded video files, as well as for screen recording when dealing with Twitter/X data

1 DOWNLOADING YOUTUBE DATA USING A PIPELINE – L. TRONCONE 2

2 DOWNLOADING TIKTOK DATA USING A PIPELINE – L. TRONCONE 7

3 DOWNLOADING AND ALIGNING TWITCH DATA USING A PIPELINE – A. ROBERT 11

3.1 INSTALLATION OF DEPENDENCIES 11
3.2 DOWNLOADING TWITCH DATA 12
3.3 AUDIO TRANSCRIPTION 14
3.4 ALIGNMENT OF CHAT AND AUDIO TRANSCRIPTION 15
3.5 ADDITIONAL TOOLS 17

4 DOWNLOADING TWITTER / X DATA USING FREE SOFTWARE – S. JEON 20

4.1 EDITING A VIDEO WITH VIDEOPROC VLOGGER 20
4.2 SCREEN RECORDING WITH OBS STUDIO 20
4.3 AUTOMATIC TRANSCRIPTION OF AN AUDIO FILE WITH SPEECHMATICS 21
4.4 AUTOMATIC TRANSCRIPTION WITH ASR_BATCH.PY 22

1 Downloading YouTube data using a pipeline – L. Troncone

YouTube data is to be processed through the code available at the following link: https://colab.research.google.com/drive/11K3lSMbrSceYztb3z1mM-PSpUu_h7BE_?usp=sharing

Through this pipeline, accessible through the tool Google CoLab, you can obtain the following files:

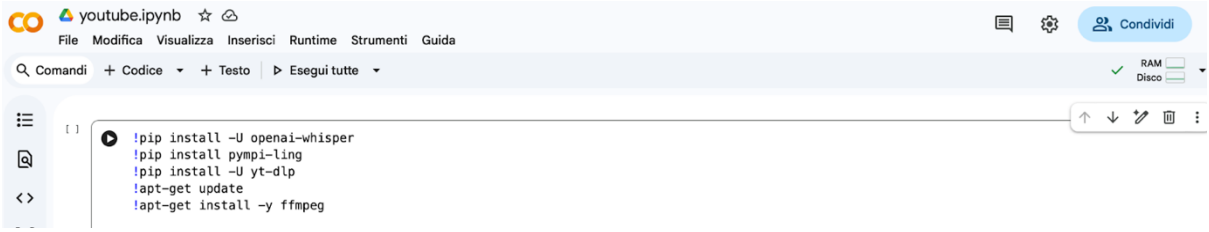
- entire video (.mp4),
- chosen segments (.mp4),
- their audio (.wav),
- respective transcriptions (.eaf, compatible with ELAN),

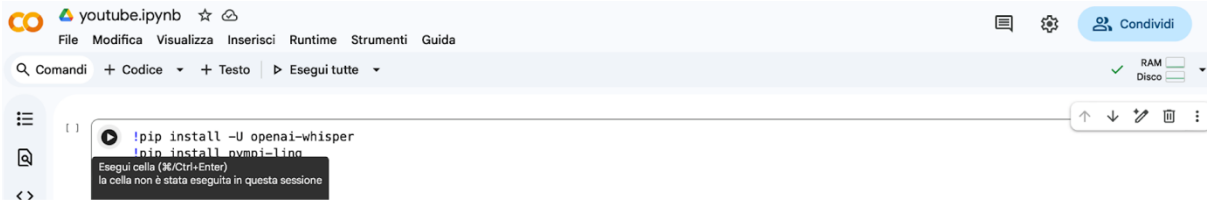
Based only on three pieces of information:

- URL of the video,
- begin time (in seconds) of the segments,
- end time (in seconds) of the segments.

Please find here a sample of the process, displayed through screenshots.

1) Run the first cell to install the packages required for the session using the play button.

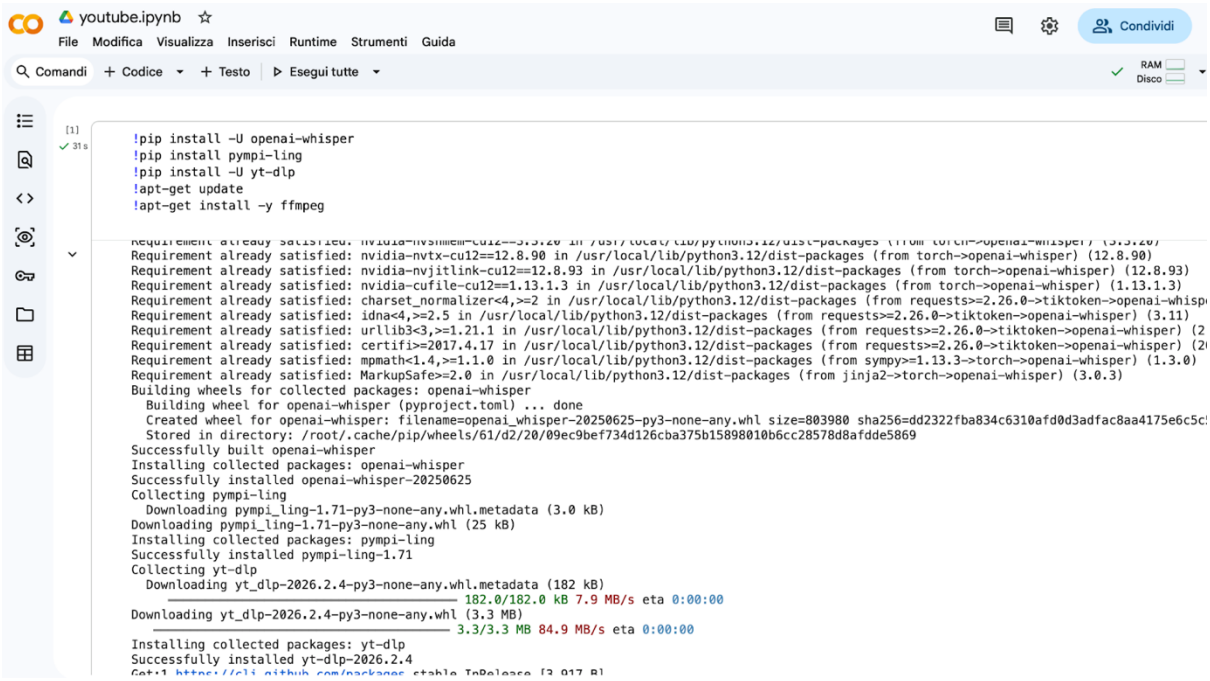




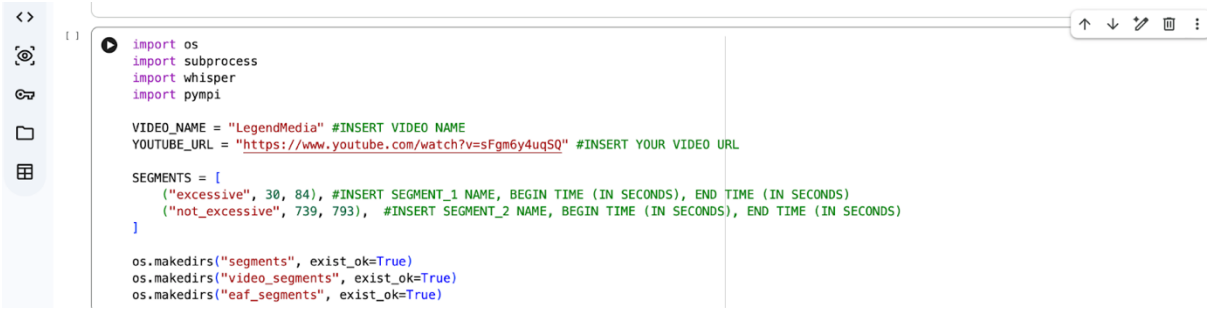
2) Cell in progress.



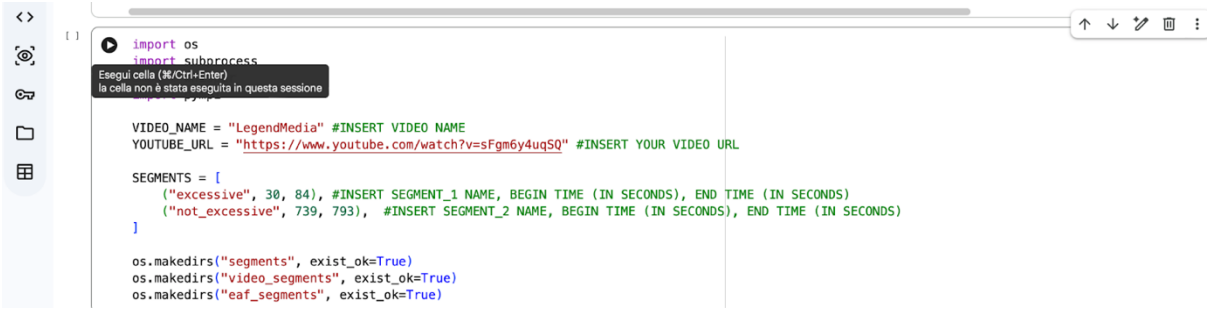
3) Successfully executed cell, indicated by a green check mark.



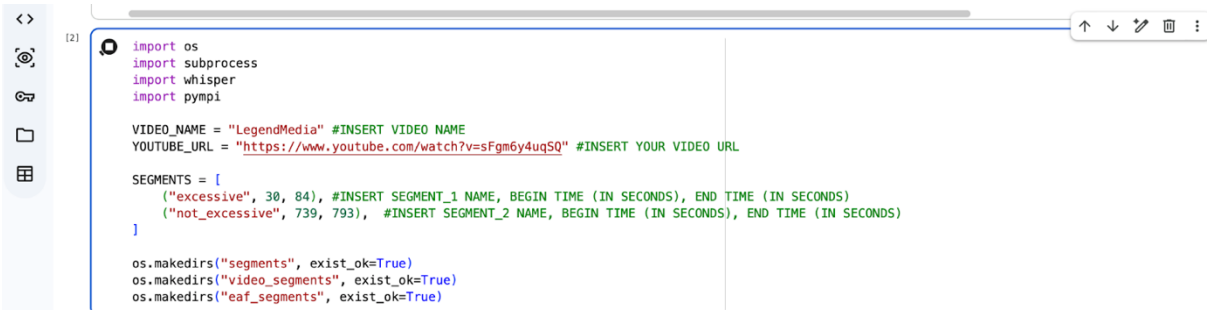
4) In this cell you will need to insert a name for your video, its URL, and the names and timings of the segments you are interested in.
In this picture, the name of the video chosen is *LegendMedia*, and the URL is https://www.YouTube.com/watch?v=sFgm6y4uqSQ.
The names for the two segments in the picture are *excessive* and *not_excessive*, and their timings are respectively 30-84 seconds, and 739-793 seconds.
Please keep in mind that to pass from minutes to seconds you have to multiply by 60 (3 minutes = 3*60 = 180 seconds), and for hours you have to multiply by 3600 (3 hours = 3*3600 = 10800).



5) Run this cell.



6) These boxes are concerned with the definition of all our variables



```
import os
import subprocess
import whisper
import pympi

VIDEO_NAME = "LegendMedia" #INSERT VIDEO NAME
YOUTUBE_URL = "https://www.youtube.com/watch?v=sFgm6y4uqSQ" #INSERT YOUR VIDEO URL

SEGMENTS = [
    ("excessive", 30, 84), #INSERT SEGMENT_1 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
    ("not_excessive", 739, 793),  #INSERT SEGMENT_2 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
]

os.makedirs("segments", exist_ok=True)
os.makedirs("video_segments", exist_ok=True)
os.makedirs("eaf_segments", exist_ok=True)
```

7) Now we can move on to the code for processing of videos. This cell is needed for the code to download the video you chose.



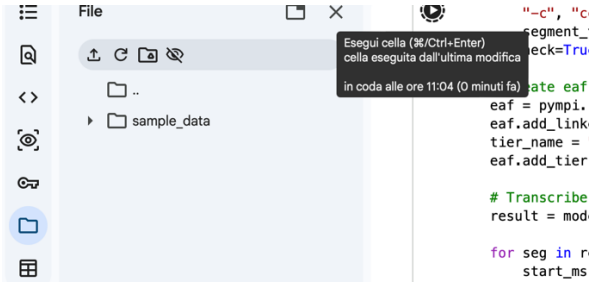
```
# 1. Download entire video
video_file = f"{VIDEO_NAME}.mp4"
subprocess.run([
    "yt-dlp",
    "-f", "mp4",
    "-o", video_file,
    YOUTUBE_URL
], check=True)
```

8) Once the video is downloaded, you will find it here:

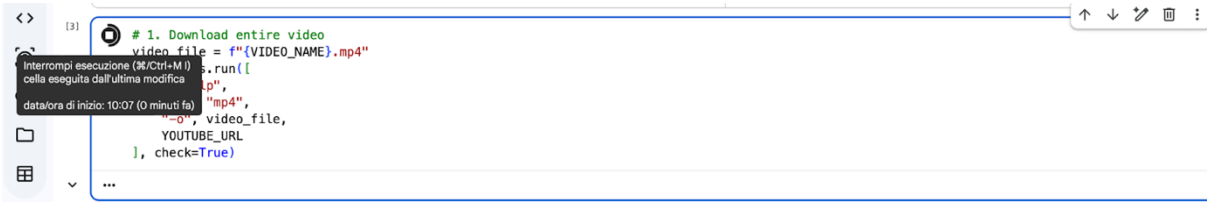


```
    "-o", video_file,
    YOUTUBE_URL
], check=True)
```

9) Click on the folder icon and you will get this environment. Here you will find all your files to provide the Sens-Ex project.

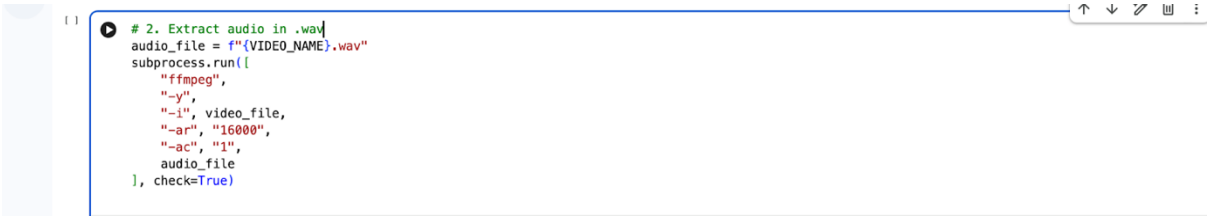


10) Run the download cell.



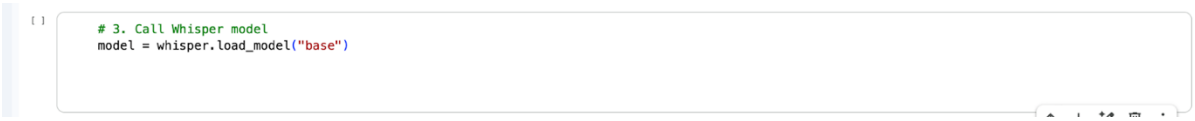

```
# 1. Download entire video
video_file = f"{VIDEO_NAME}.mp4"
    "mp4",
    "-o", video_file,
    YOUTUBE_URL
], check=True)
```

11)The next step is the extraction of the .wav audio file from the video. The transcription will be based on this file. Run the cell.



```
# 2. Extract audio in .wav
audio_file = f"{VIDEO_NAME}.wav"
subprocess.run([
    "ffmpeg",
    "-y",
    "-i", video_file,
    "-ar", "16000",
    "-ac", "1",
    audio_file
], check=True)
```

12)Now we turn to the import of the Whisper model. Whisper is the software by OpenAI which we will use to create an automatic transcription of the audio. Run the cell.



```
# 3. Call Whisper model
model = whisper.load_model("base")
```

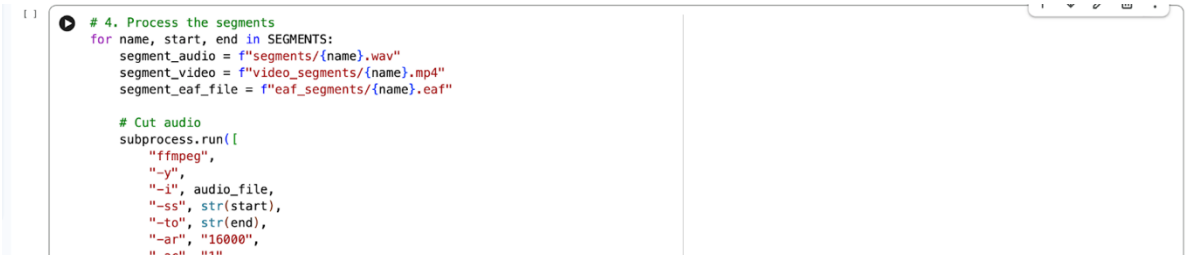
13)Once the import is complete, you will see a black bar to 100% below the cell.



```
# 3. Call Whisper model
model = whisper.load_model("base")
```

```
100%|| 139M/139M [00:00<00:00, 195MiB/s]
```

14)Next is the processing of segments, cutting off audio and transcription of all media files.



```
# 4. Process the segments
for name, start, end in SEGMENTS:
    segment_audio = f"segments/{name}.wav"
    segment_video = f"video_segments/{name}.mp4"
    segment_eaf_file = f"eaf_segments/{name}.eaf"

    # Cut audio
    subprocess.run([
        "ffmpeg",
        "-y",
        "-i", audio_file,
        "-ss", str(start),
        "-to", str(end),
        "-ar", "16000",
```



```
result = model.transcribe(segment_audio)

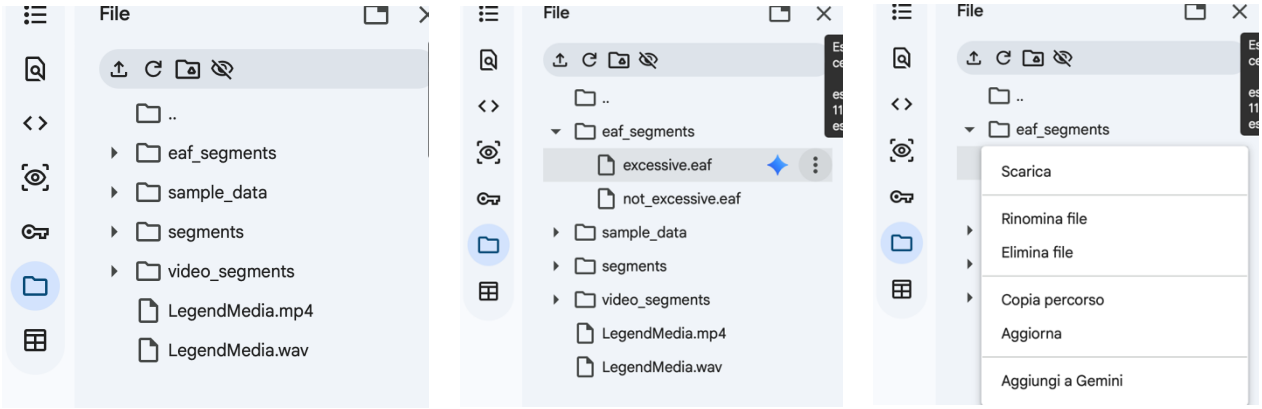
for seg in result["segments"]:
    start_ms = int(seg["start"] * 1000)
    end_ms = int(seg["end"] * 1000)
    text = seg["text"].strip()
    eaf.add_annotation(tier_name, start_ms, end_ms, text)

# Save eaf file
eaf.to_file(segment_eaf_file)
print(f"✅ Segmento '{name}' completato: {segment_video}, {segment_eaf_file}")
```

```
✅ Segmento 'excessive' completato: video_segments/excessive.mp4, eaf_segments/excessive.eaf
```

```
print("🎉 Tutti i segmenti processati!")
```

15) Finally, you can access all the generated files in the list on the left and download them using the three-dot menu.



Here *scarica* for Download.

2 Downloading TikTok data using a pipeline – L. Troncone

TikTok data should be processed through the following CoLab code:
https://colab.research.google.com/drive/1zjfH7vtEWzaPjVSAaVnIwRfqjlEPblGJ?usp=sharing

It is important however to keep in mind the best practices for an ethical workflow in research when dealing with personal (an potentially sensible) data, like social media data. It is recommended to ask for the research API to TikTok : https://developers.tiktok.com/products/research-api/.
For compliance, see: https://www.tiktok.com/legal/page/global/terms-of-service-research-api/en.
That said, let's start.

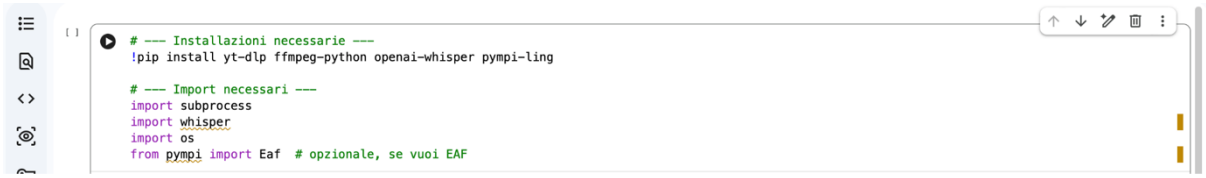
Through this pipeline, accessible through the tool Google CoLab, you can obtain the following files

- video (.mp4),
- audio (.wav),
- respective transcriptions (.eaf, compatible with ELAN, and .txt),

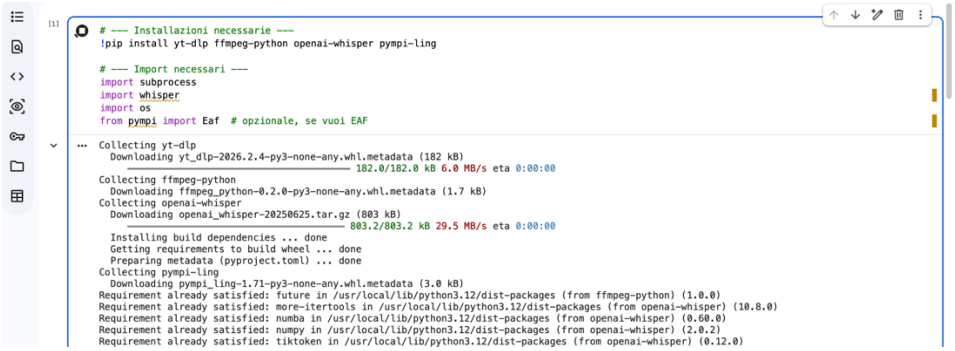
based only on the URL of the video.

Please find here a sample of the process, displayed through screenshots.

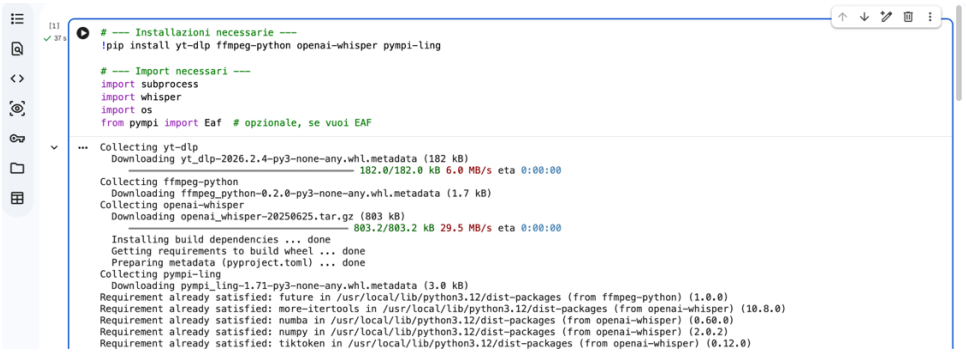
1) Run the first cell to install the required packages using the play button.



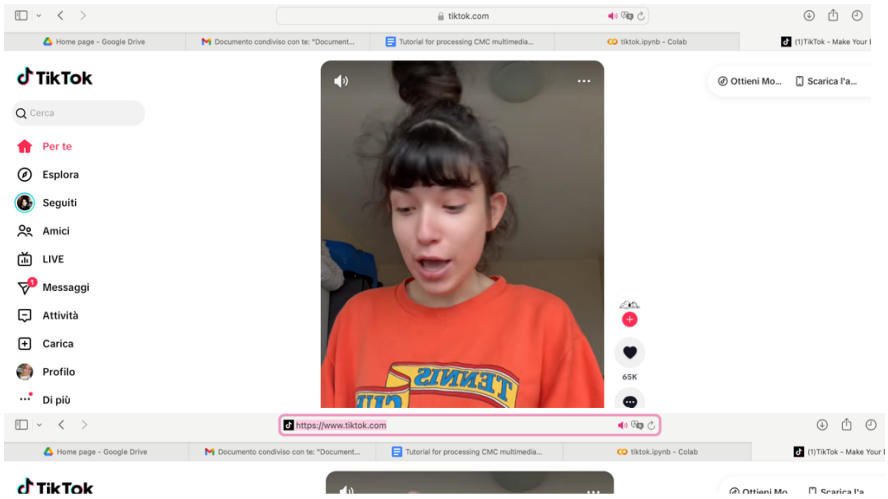
2) Cell in progress.



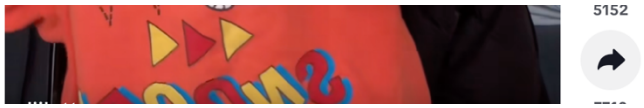
3) Successfully executed cell, indicated by a green check mark.



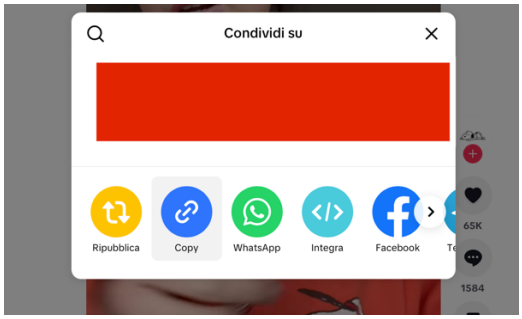
4) Now, before going on with the cells, we need to copy the URL of the TikTok video we want to process. To do that, find the video on TikTok.



5) As you will notice, TikTok URLs for specific video is not found on the top so you need to search for it in the metadata of the video and press on the *share* button.



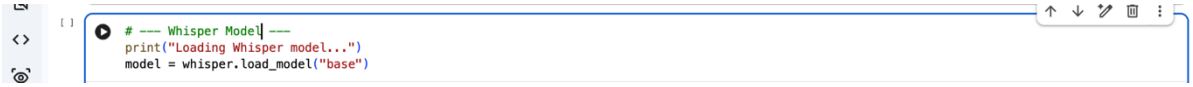
6) You will get this view. Click on copy. Now you have the URL you need.



7) Go back to Google CoLab. In the next cell, you need to paste the URL you just copied.



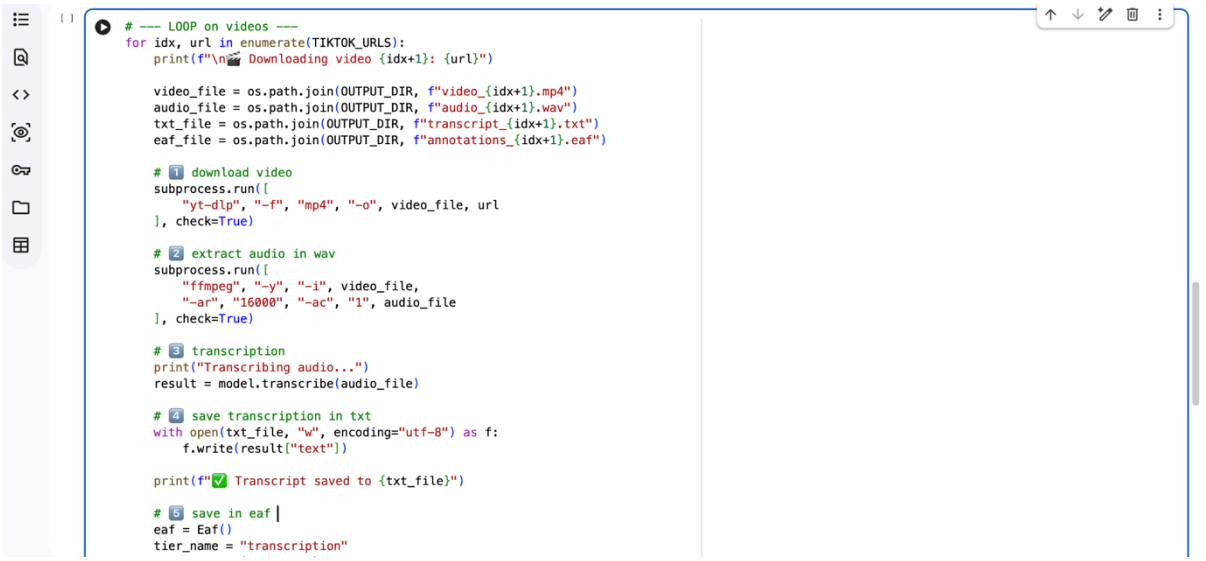
8) Next cell is about the loading of the Whisper Model. Whisper is a model developed by OpenAI that we will use to automatically transcribe the video/audio.



9) Run it. Once it's done, you will get a black 100% bar underneath the cell.



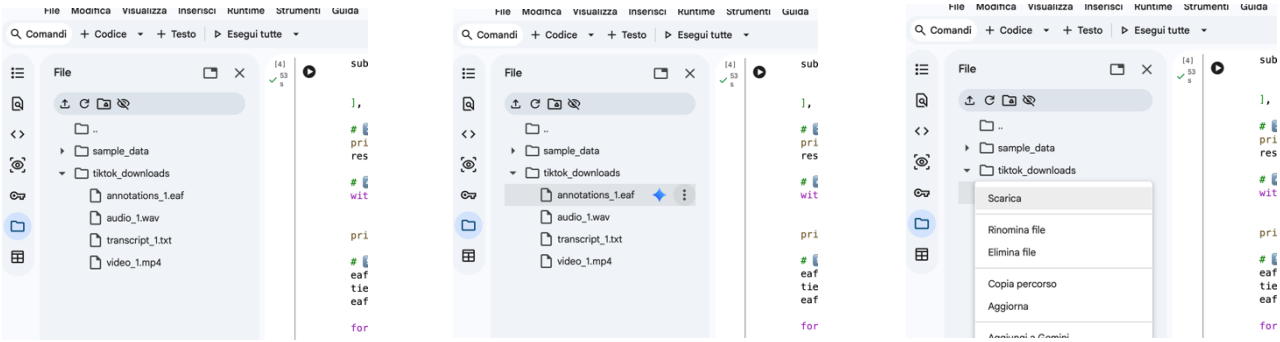
10) We can now move on to the processing of the video. Run the cell.



11) To get all the processed files, look at the left side of the screen. You will see a folder button.



12) Finally, you can access all the generated files in the *tiktok_downloads* subfolder and download them using the three-dot menu.



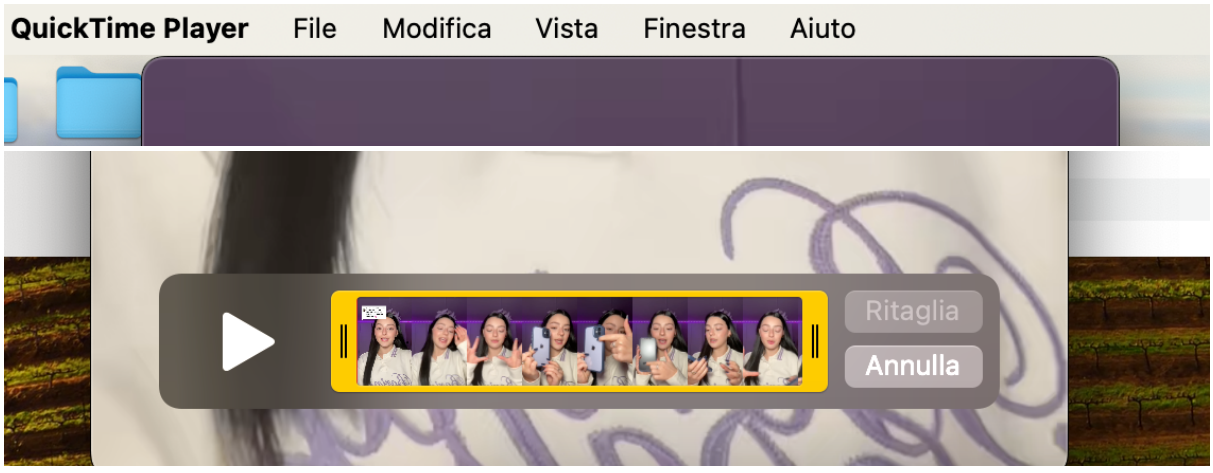
Here *scarica* for download.

Keep in mind that for each tiktok video to process, you need to delete all data downloaded previously.

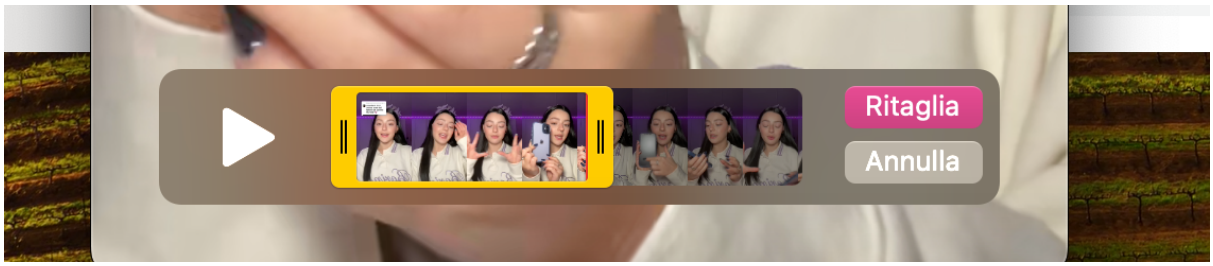
13)If the video is too long, you will need to cut it through some video editor like

- On Windows: the **photo editor** for recent versions, or **windows movie maker** for previous versions
- On MacOS: on the video player (see picture below)

14) On QuickTime Player (MacOS), in the upper menu, click on *Modify* (here *Modifica*) , there you will find the *cut* button – you should get the same view as below.



15)Choose the segment you are interested in, and press the *cut* button (here *Ritagglia*).

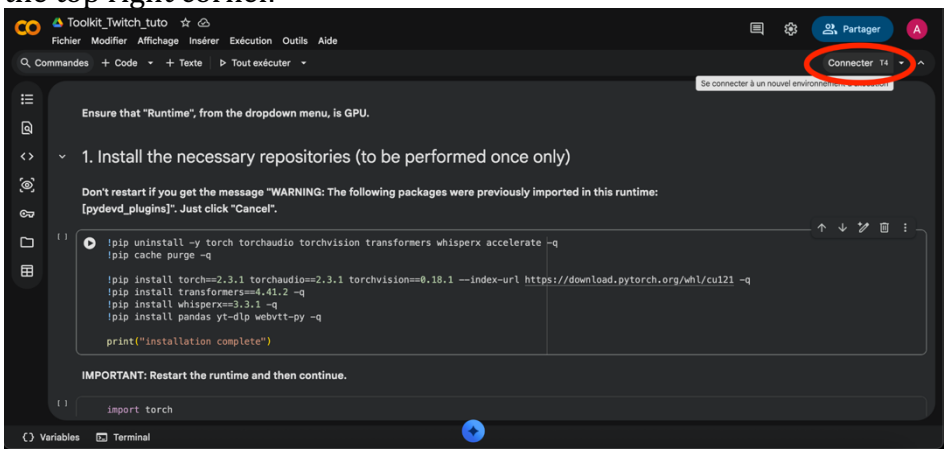


3 Downloading and aligning Twitch data using a pipeline – A. Robert

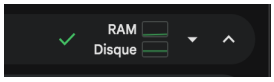
The toolkit was developed by S. Coats (2024) and adapted by A. Robert (2025). It must be run in the Google Colab available at the following link. It includes several steps:

- Installation of dependencies
- Downloading Twitch data
- Transcription of audio data
- Alignment of chat and audio transcription
- Additional tools

Before running the different cells, make sure to connect to the GPU using the button in the top right corner.



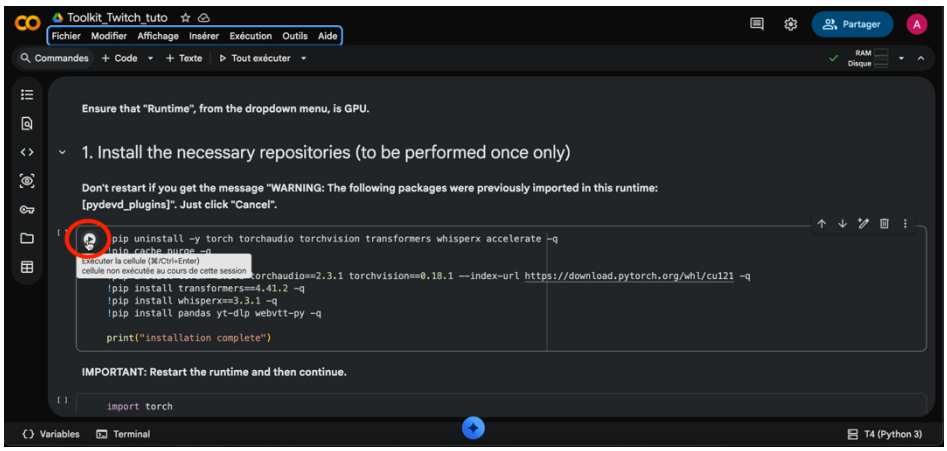
The button should then appear as follows.



Important: during execution, a pop-up window may ask you to restart the session. Do not restart it unless explicitly instructed.

3.1 Installation of dependencies

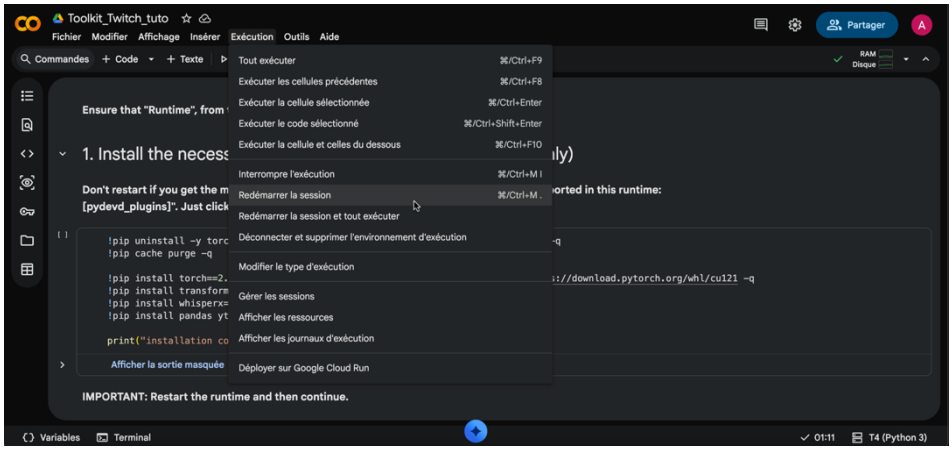
STEP 1: Run the first cell using the execution button.



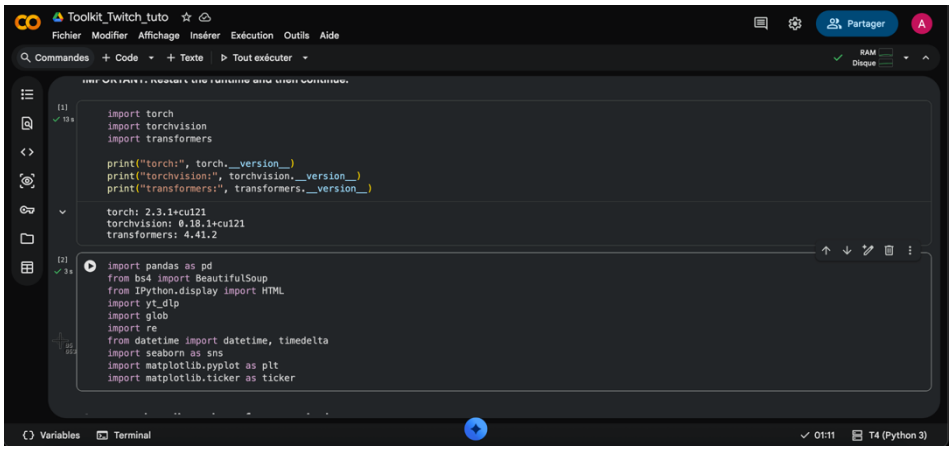
A green indicator means the cell has been successfully executed, a red indicator means it has not.



STEP 2: Restart the session: Menu → Runtime > Restart session

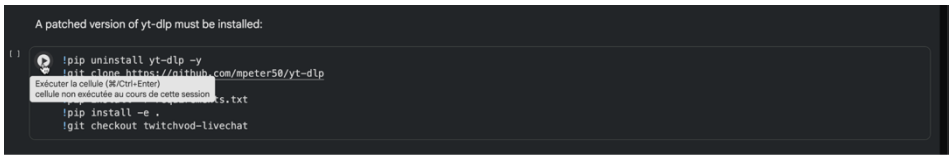


STEP 3: Run the next two cells.

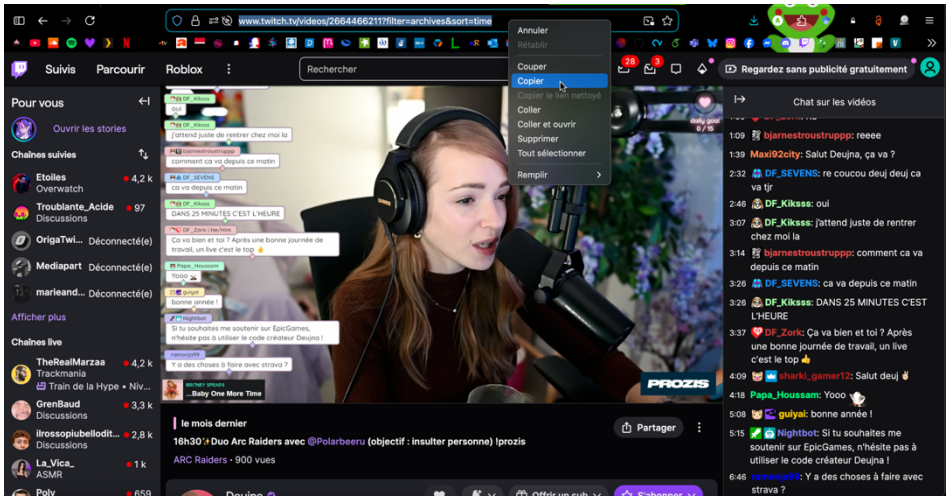


3.2 Downloading Twitch data

STEP 4: Install *yt-dlp* to download audio from Twitch VODs by running the following cell.

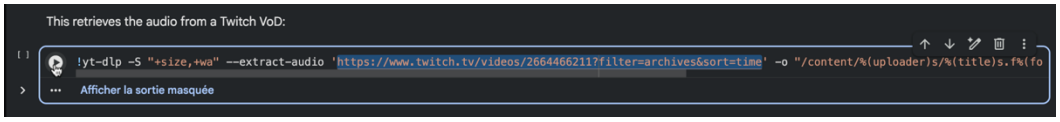


STEP 5: aller sur la plateforme Twitch, sélectionner la vidéo à télécharger. Une fois sélectionnée, copier l'URL de la vidéo:

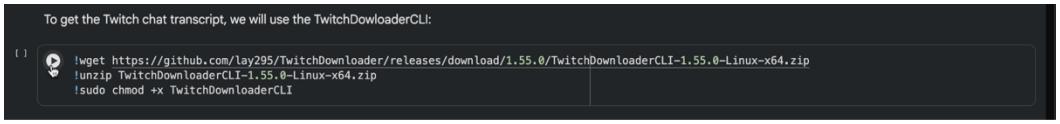


Return to Google Colab, paste the URL into the corresponding cell (between the quotation marks), and run it.

(To extract only part of the audio, see **Step 17B**.)



STEP 6: Install *TwitchDownloaderCLI* to download chat data.

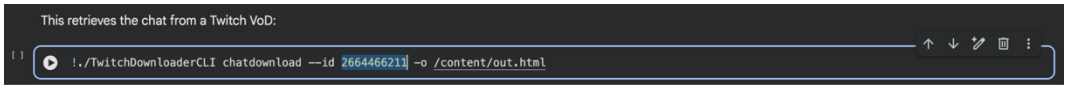


STEP 7: From the Twitch VOD URL, copy the video ID (highlighted in the tutorial).



Paste this ID into the corresponding cell and run it to download the chat.

(To extract only part of the chat, see Step 17C.)



STEP 8: Run the next two cells to: (a) structure chat data:



(b) verify the chat file (it should look like the picture below):



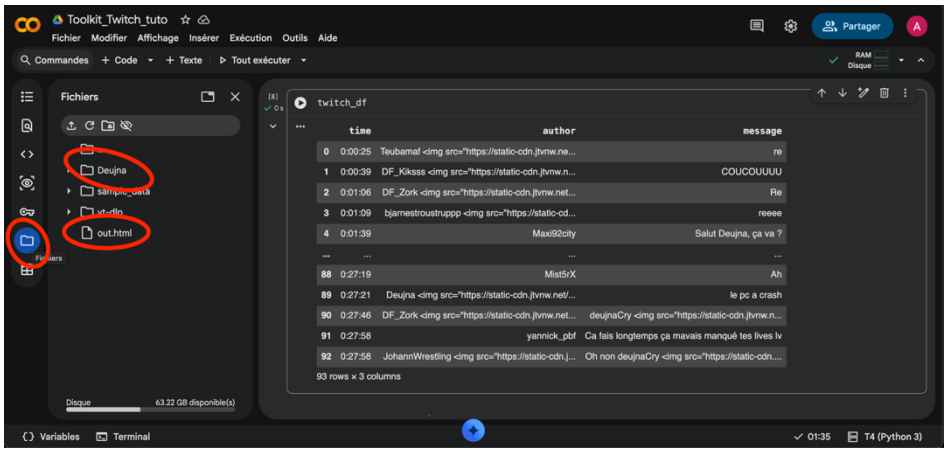
twitch_df

	time	author	message
0	0:00:25	Teubamaf <img src="https://static-cdn.jtvnw.ne...	re
1	0:00:39	DF_Kiksss <img src="https://static-cdn.jtvnw.n...	COUCOUUUU
2	0:01:06	DF_Zork <img src="https://static-cdn.jtvnw.net...	Re
3	0:01:09	bjarnestroustruppp <img src="https://static-cd...	reeee
4	0:01:39	Maxi92city	Salut Deujna, ça va ?
...	...	...	...
88	0:27:19	Mist5rX	Ah
89	0:27:21	Deujna <img src="https://static-cdn.jtvnw.net/...	le pc a crash
90	0:27:46	DF_Zork <img src="https://static-cdn.jtvnw.net...	deujnaCry <img src="https://static-cdn.jtvnw.n...
91	0:27:58	yannick_pbf	Ca fais longtemps ça mavais manqué tes lives lv
92	0:27:58	JohannWrestling <img src="https://static-cdn.j...	Oh non deujnaCry <img src="https://static-cdn....

93 rows × 3 columns

The file named **out.html** is available in the left folder.

Make sure to download it as Google CoLab does not store any files.



Nb: audio is available in the file named as the streamer (here “Deujna”).

3.3 Audio transcription

STEP 9: Run the first cell of this section to transcribe the audio.



3. Transcription of the audio with WhisperX

```
import whisperx
import torch
import glob
import pandas as pd
import gc

device = "cuda" if torch.cuda.is_available() else "cpu"

audio_file = glob.glob("/content/*/*.m4a")

model = whisperx.load_model(
    "small",
    device,
    compute_type="int8",
    language="fr"
)

audio = whisperx.load_audio(audio_file[0])

result = model.transcribe(
    audio,
    batch_size=2
)
```

The check that the transcription was correctly executed.

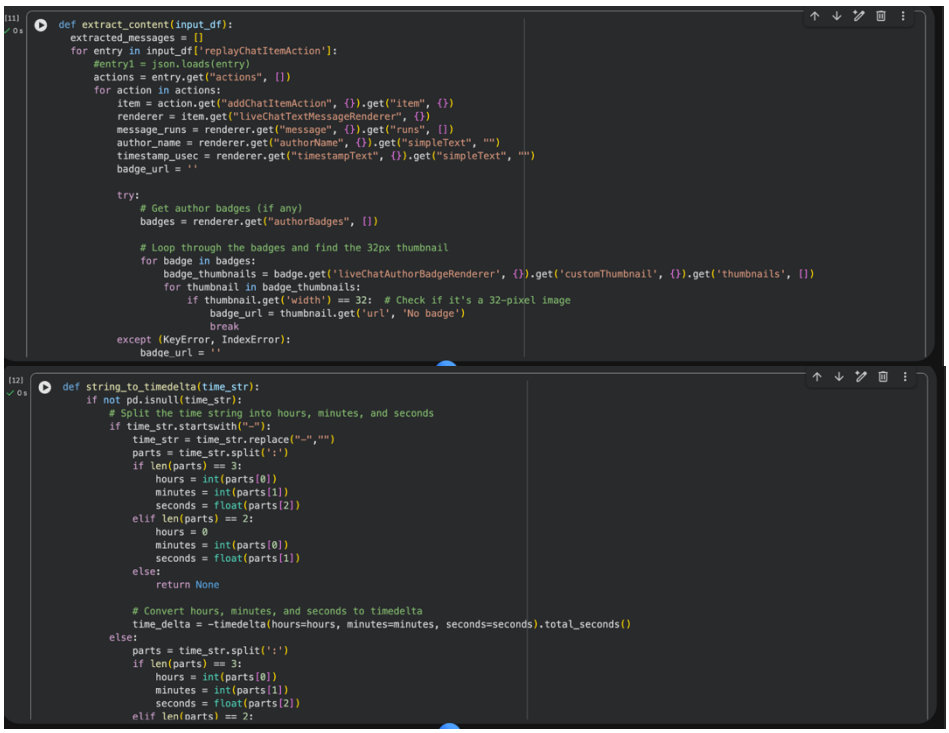


whisper_df1

	time	text
0	157.624	Allô ?
1	159.504	Vous êtes fatigué ou quoi la team ?
2	162.605	Vous êtes plutôt fatiguée ou pas ?
3	167.250	Comment ça va ?
4	168.271	Hello hello !
...	...	...
439	1578.942	Il est où ?
440	1579.363	Ah, il est au-dessus de moi.
441	1580.726	Ah, il me voit à travers le bus pour baisser q...
442	1588.320	T'arrives à t'en sortir ou pas ?
443	1590.243	Non, non, je suis morte là.

444 rows × 2 columns

STEP 10: Run the next two cells to structure the data.



```
def extract_content(input_df):
    extracted_messages = []
    for entry in input_df['replayChatItemAction']:
        #entry1 = json.loads(entry)
        actions = entry.get("actions", [])
        for action in actions:
            item = action.get("addChatItemAction", {}).get("item", {})
            renderer = item.get("liveChatTextMessageRenderer", {})
            message_runs = renderer.get("message", {}).get("runs", [])
            author_name = renderer.get("authorName", {}).get("simpleText", "")
            timestamp_usec = renderer.get("timestampText", {}).get("simpleText", "")
            badge_url = ''

            try:
                # Get author badges (if any)
                badges = renderer.get("authorBadges", [])

                # Loop through the badges and find the 32px thumbnail
                for badge in badges:
                    badge_thumbnails = badge.get('liveChatAuthorBadgeRenderer', {}).get('customThumbnail', {}).get('thumbnails', [])
                    for thumbnail in badge_thumbnails:
                        if thumbnail.get('width') == 32:  # Check if it's a 32-pixel image
                            badge_url = thumbnail.get('url', 'No badge')
                            break
            except (KeyError, IndexError):
                badge_url = ''
```

```
def string_to_timedelta(time_str):
    if not pd.isnull(time_str):
        # Split the time string into hours, minutes, and seconds
        if time_str.startswith("-"):
            time_str = time_str.replace("-","")
            parts = time_str.split(':')
            if len(parts) == 3:
                hours = int(parts[0])
                minutes = int(parts[1])
                seconds = float(parts[2])
            elif len(parts) == 2:
                hours = 0
                minutes = int(parts[0])
                seconds = float(parts[1])
            else:
                return None

            # Convert hours, minutes, and seconds to timedelta
            time_delta = -timedelta(hours=hours, minutes=minutes, seconds=seconds).total_seconds()
        else:
            parts = time_str.split(':')
            if len(parts) == 3:
                hours = int(parts[0])
                minutes = int(parts[1])
                seconds = float(parts[2])
            elif len(parts) == 2:
```

3.4 Alignment of chat and audio transcription

STEP 11: Merge chat and transcription files by running the two next cells



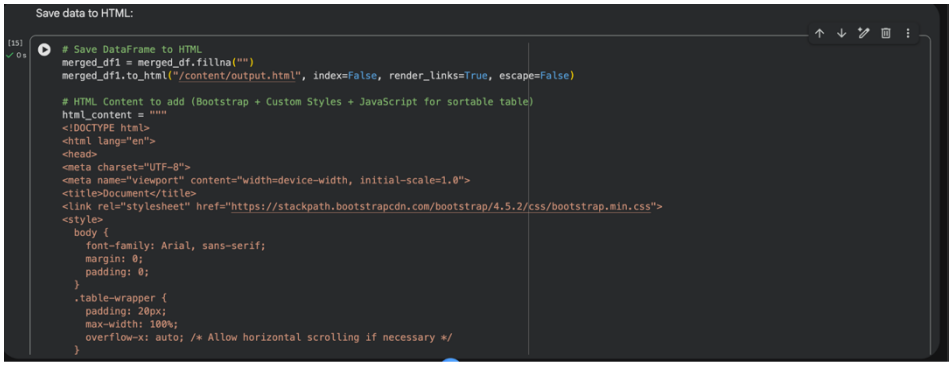
```
4. Merge the chat and transcript data

Merge:

twitch_df["time"] = twitch_df["time"].apply(lambda x: string_to_timedelta(str(x)))

merged_df = pd.merge_ordered(whisper_df1, twitch_df, on='time')
merged_df = merged_df[merged_df["time"].notnull()].sort_values(by="time")
```

STEP 12: Export the alignment in HTML format

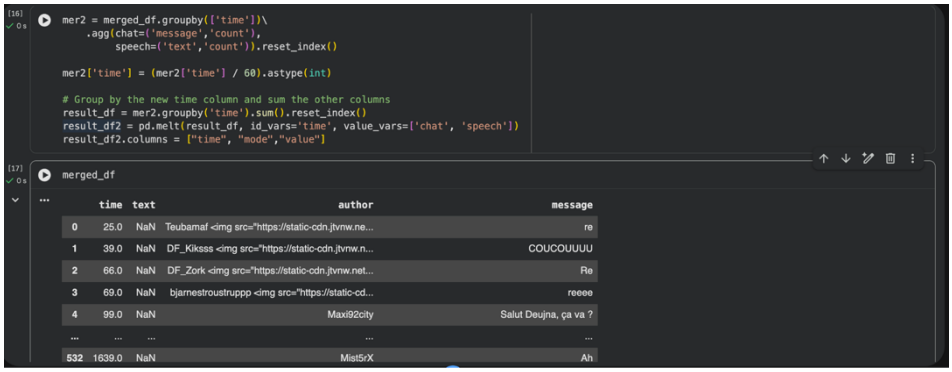


```
Save data to HTML:

# Save DataFrame to HTML
merged_df1 = merged_df.fillna("")
merged_df1.to_html("/content/output.html", index=False, render_links=True, escape=False)

# HTML Content to add (Bootstrap + Custom Styles + JavaScript for sortable table)
html_content = """
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
<style>
  body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
  }
  .table-wrapper {
    padding: 20px;
    max-width: 100%;
    overflow-x: auto; /* Allow horizontal scrolling if necessary */
  }
```

STEP 13: Finalize the alignment and verify the result.



```
mer2 = merged_df.groupby(['time'])\
    .agg(chat=('message','count'),
         speech=('text','count')).reset_index()

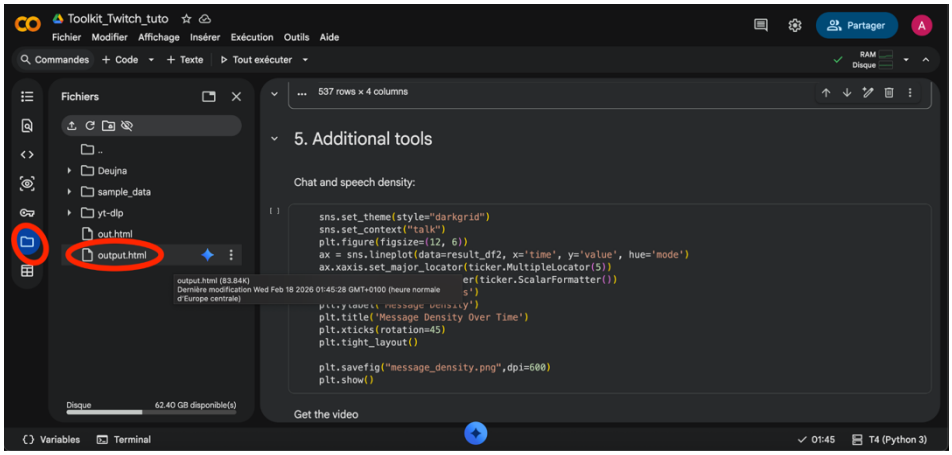
mer2['time'] = (mer2['time'] / 60).astype(int)

# Group by the new time column and sum the other columns
result_df = mer2.groupby('time').sum().reset_index()
result_df2 = pd.melt(result_df, id_vars='time', value_vars=['chat', 'speech'])
result_df2.columns = ["time", "mode","value"]
```

```
merged_df
```

	time	text	author	message
0	25.0	NaN	Teubamaf <img src="https://static-cdn.jtvnw.ne...	re
1	39.0	NaN	DF_Kiksss <img src="https://static-cdn.jtvnw.n...	COUCOUUUU
2	66.0	NaN	DF_Zork <img src="https://static-cdn.jtvnw.net...	Re
3	69.0	NaN	bjarnestroustruppp <img src="https://static-cd...	reeee
4	99.0	NaN	Maxi92city	Salut Deujna, ça va ?
...	...	...	...	...
532	1639.0	NaN	Miist5rX	Ah

At this stage, all steps are complete and the aligned Twitch data (audio and chat) have been successfully generated. The file named **output.html** is availablebale for download in the same folder as the chat file **(please make sure to download it as Google Collabs does not store any files)**.



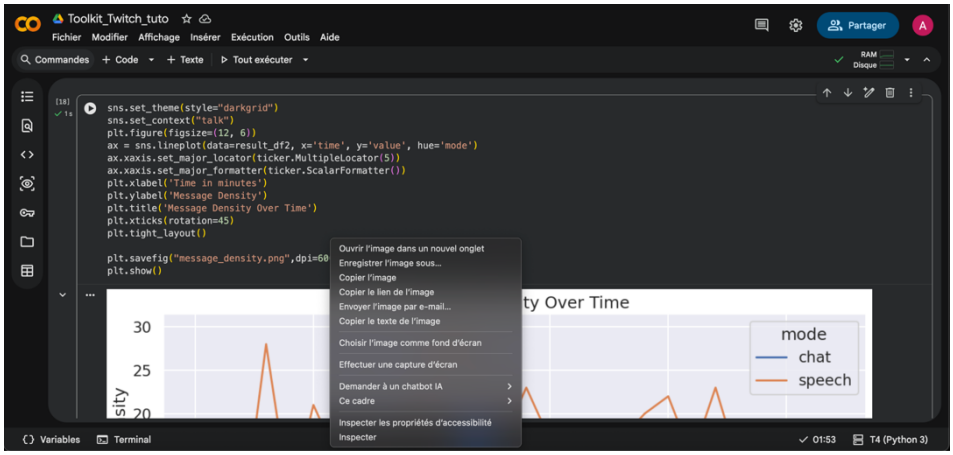
The HTML file contains 4 columns: (i) Time ; (ii) Audio transcription ; (iii) Usernames ; (iv) Chat messages. Chat and transcription are aligned over time.
Below an example of the output you should obtain:

time	text	author	message
25.000		Teubamaf	re
39.000		DF_Kiksss	COUCOUUUU
66.000		DF_Zork	Re
69.000		bjarnestroustruppp	reeee
99.000		Maxi92city	Salut Deujna, ça va ?
152.000		DF_SEVENS	re coucou deuj deuj ca va tjr
157.624	Allô ?		
159.504	Vous êtes fatigué ou quoi la team ?		
162.605	Vous êtes plutôt fatiguée ou pas ?		
166.000		DF_Kiksss	oui
167.250	Comment ça va ?		
168.271	Hello hello !		

3.5 Additional tools

The following additional tools can be used to complement the data (see the cell under section 5)

STEP 14: **To obtain a graph visualizing chat density and turn-taking**, simply run the following cell:



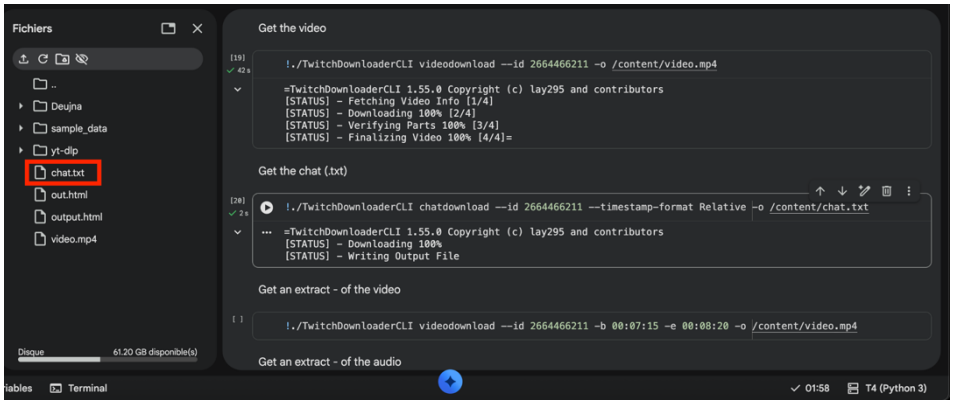
You can save the image as a .png file by right-clicking on it and selecting "*Save image as*".

STEP 15: **To obtain the video in .mp4 format**, copy the VOD identifier from the URL and paste it after "id" (as in step 7) in the following cell, then run the cell (to obtain only a segment of the video, see step 17a)



This video is available in the folder under the name "*video.mp4*" (**make sure to download it as Google CoLab does not save files**).

STEP 16: **To obtain the chat in .txt format**, copy the VOD identifier from the URL and paste it after "id" (as in step 7) in the following cell, then run the cell:



the chat is available in the folder under the name "*chat.txt*" (**make sure to download it as Google CoLabs does not save files**).

STEP 17: **To download only a portion of the VOD (chat, video, and audio)**, you can use the following three cells instead of some of the previous steps:

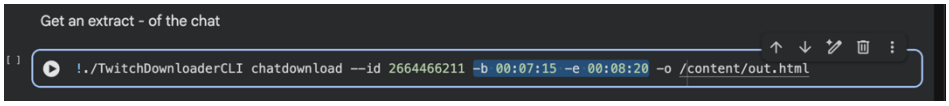
- **A [instead of Step 15]: To obtain a portion of the video**, modify the time values in the following cell (for example, in the image below, the highlighted parts correspond to a selection between 7 minutes 15 seconds and 8 minutes 20 seconds), then run the cell.



- **B [instead of Step 5]: To obtain a portion of the audio**, modify the time values in the following cell (for example, in the image below, the highlighted parts correspond to a selection between 7 minutes 15 seconds and 8 minutes 20 seconds), then run the cell.

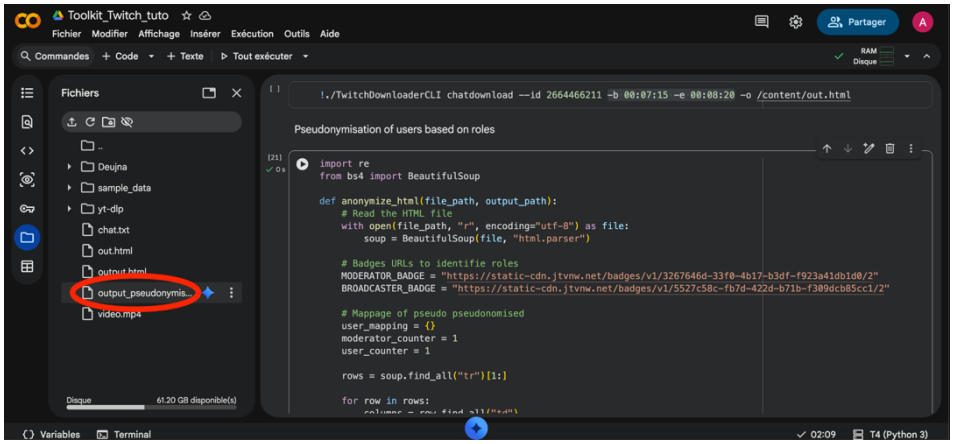


- **C [instead of Step 7]: To obtain a portion of the chat**, modify the time values in the following cell (for example, in the image below, the highlighted parts correspond to a selection between 7 minutes 15 seconds and 8 minutes 20 seconds), then run the cell.



Important: When working with excerpts, it is crucial to select exactly the same time spans for the video, audio, and chat. You must also follow the execution order of the cells and steps carefully (for example, once Step 7 is replaced by Step 17C, continue with Step 18).

To generate a pseudonymized version of the aligned files, run the final cell:

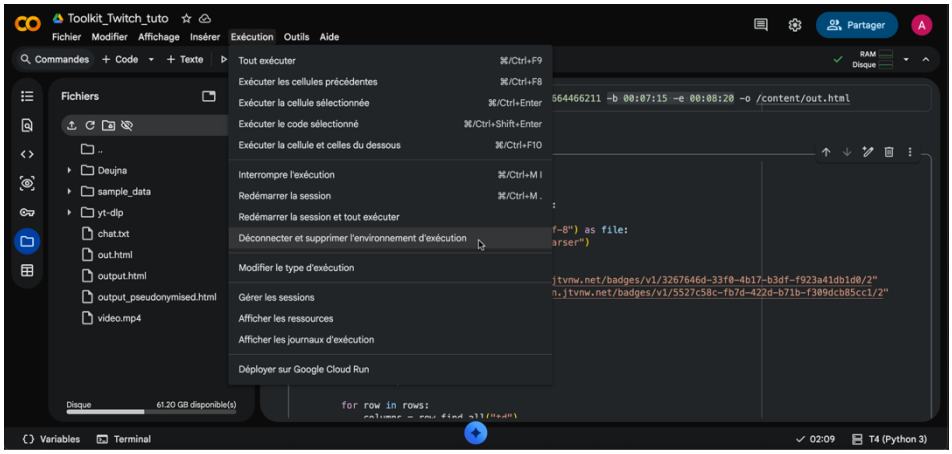


The pseudonymized file will be available in the folder under the name "output_pseudonymisé.html" (**make sure to download it, as Google Colab does not save files permanently**). It should appear as follows:

time	text	author	message
10.000		User1	bgrLOUTRE Bonjour avamindLuv
12.000		User2	avamindBONJ avamindLOVE
22.340	Oh non, pas un arc, hein.		
24.000		User3	avamindLuv avamindLuv avamindLuv avamindLuv avamindLuv
29.000		User4	avamindBONJ avamindBONJ avamindLuv avamindLuv
30.572	Qu'est-ce que... Oh non, hein.		
32.135	Ah, faut arrêter avec les arcs, hein.		
33.000		User5	avamindBONJ acastC1 acastAcastC2 acastC3 avamindLuv
33.737	C'est fini, ça.		
40.000		User6	bonjour avamindLuv

Important: Before downloading, transcribing, and aligning new Twitch data, it is necessary to reset the environment.

To do so, go to: *Runtime > Disconnect and delete runtime.*



4 Downloading Twitter / X data using free Software – S. Jeon

When multimedia data are available as raw video files, it is recommended to use free software. In this subsection, three free tools are presented. They have been selected for their compatibility with the three main operating systems (Windows, Mac, Linux).

- **Video editing software:** VideoProc Vlogger
  https://www.videoproc.com/video-editing-software/guide-free-mac/cut-split.htm?ttsoft=vpvmac-1.4-tips-trial
- **Automatic audio transcription tool:** Speechmatics
  https://www.speechmatics.com

For posts on X, we proceed by screen recording in order to capture the embedded multimedia data.

- **Screen recording tool:** OBS Studio
  https://obsproject.com/fr

This tutorial is designed as a ready-to-use guide tailored to the needs of the project, focusing specifically on the functionalities required for processing multimedia data, namely:

- Editing a video to extract relevant segments
- Automatically transcribing these segments for inclusion in the corpus
- Recording the screen to archive and collect Twitter/X posts

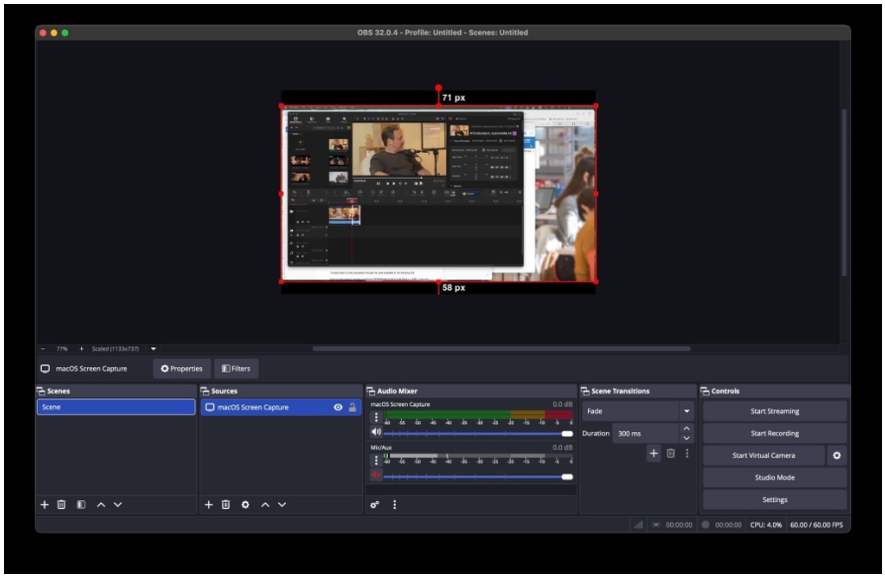
You can find a tutorial for the software VideoProc Vlogger in video format on YouTube at the following link:
https://youtu.be/1GRFJ8KwxKQ

4.1 Editing a video with VideoProc Vlogger

Install the software.

4.2 Screen recording with OBS studio

Install the software.
Once OBS is installed, launch the application. The main interface appears as shown below.



It is necessary to precisely define the screen capture area so that the entire content of the X post is clearly visible.
To do so, click on the screen preview displayed in the main interface window. Make sure that the selected screen is outlined in red, then click on the *Properties* button.
Three options are available: *Display Capture*, *Window Capture*, *Application Capture*. For example, you can select *Window Capture*, then choose the web browser displaying the target post. It is also possible to hide the cursor by unchecking *"Show cursor"*.
In the **Audio Mixer** panel: once the capture area is defined, ensure that system audio (macOS Screen Capture or Desktop Audio) is enabled and that the microphone is disabled.
In the **Controls** panel: before starting the recording, check the output file format. Click on *Settings*. In the *Output* tab, go to the *Recording* section, then select *Hybrid MP4 (.mp4)* as the recording format. Click *OK* to confirm.

At this stage, all required settings are configured: capture area defined, output format set to MP4, microphone disabled, and system audio enabled.

Starting the recording
In the browser window displaying the X post, click the play button.
In the **Controls panel**, click *Start Recording*. During recording, the button turns blue and displays *Stop Recording*. When the video ends, click *Stop Recording*. Then check the location of the saved file, which is displayed at the bottom of the interface.

4.3 Automatic transcription of an audio file with Speechmatics

Go to the Speechmatics website. Create a user account and generate an API key, which will be required.
In the left panel of the interface, click on *Speech to text > File transcription*.
The *Create* window appears. Select *Batch Transcribe previously recorded media*, then click *Upload a file*.
This feature supports the following formats: aac, amr, flac, m4a, mp3, mp4, mpeg, ogg, wav, for files smaller than 1GB.

Audio extraction (convert_media_audio.py)
To reduce processing time, it is recommended to use a compact audio format such as MP3. However, WAV files are required for phonological analysis in Praat.
In this project, video files (.mp4) and audio files (.wav) are associated with the .eaf file (ELAN). This implies that video files must be converted to WAV format.
The script *convert_media_audio.py* allows this conversion. Instructions:

- Install Python
- Select a working directory and place the script and media files in it
- Open the Terminal
- Navigate to the directory using: `cd /path/to/your/folder`
- Run the script: `python convert_media_audio.py`
- Follow the instructions displayed in the Terminal

Format management in the processing pipeline: each data is represented in several formats with different functions:

- mp4 -> archiving and association with .eaf
- wav -> association with .eaf and prosodic analysis (Praat)
- mp3 -> automatic transcription
- eaf -> temporal alignment, segmentation into illocutionary units
- txt-> transcriptions

4.4 Automatic transcription with asr_batch.py

You can also use the script *asr_batch.py* to generate automatic transcriptions of audio files (automatic speech recognition).
Installation instructions for the *faster-whisper* library and execution steps are included at the beginning of the script.

(step1) In Terminal

- Install faster-whisper
  ```
  pip3 install faster-whisper
  ```
- After installation, verify:
  ```
  python3 -c "from faster_whisper import WhisperModel; print('OK')"
  ```

(step2)

- In the same directory where asr_batch.py is saved, create a folder entitled "audio_in" then place your input audio files inside

(step3) Run transcription in Terminal:

- Italian speech recognition

```
python3 asr_batch.py ./audio_in ./out --engine whisper --model medium --language it --formats txt,json
```

- French speech recognition

```
python3 asr_batch.py ./audio_in ./out --engine whisper --model medium --language fr --formats txt,json
```